

## stream2\_buildcode

# Stream FLAT-BUILD-CODE — build-verify helix\_code (Bash ok; NO Edit — propose patches as text)

Repo: /Users/milosvasic/Projects/HelixCode. Inner app `helix_code/` (module `dev.helix.code`, go 1.26). Phase 2 flatten rewrote its `go.mod` replaces to `../submodules/X`.

Pasted terminal output is REQUIRED evidence — no PASS/FAIL verdict without the actual output (anti-bluff, CLAUDE.md Rule 8/9).

Tasks: 1. `cat helix_code/go.mod` — paste the FULL replace section so I can confirm every target is `../submodules/X`. 2. Build gate (capture output): `- cd /Users/milosvasic/Projects/HelixCode/helix_code && go build ./... 2>&1 | tail -120 -cd /Users/milosvasic/Projects/HelixCode/helix_code && go vet ./... 2>&1 | tail -120` Report PASS/FAIL for each with the captured tail. 3. Inspect `helix_code/internal/cache/cache.go` and `helix_code/internal/i18n_wiring/wiring_integration_test.go` (both flagged with stale dependencies/{org}/refs). Quote the offending lines; classify each as (a) harmless comment/string literal, or (b) build-breaking import/path. 4. If anything FAILS: root-cause (stale import? missing replace? a dropped dep `doc_processor/llm_orchestrator/llm_provider/vision_engine` now only from HD?) and propose the MINIMAL patch as exact `file` :  
`line` : change. Do not apply.

Output: compact markdown (go.mod replace block, build result+output, vet result+output, the 2 file verdicts, patch proposals).